How-To Library

Informatica

Archive and Retirement Job Steps in Data Archive

© Copyright Informatica LLC 2019, 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html

Abstract

This document describes the internals of the steps involved during Archive and Retirement jobs scheduled in the Data Security Group (DSG) Data Archive Service. The document considers the source and target databases as Oracle databases in the description of Online Archive jobs.

Supported Versions

- Data Archive 6.5
- Data Archive 6.4.4
- Data Archive 6.4.3 HotFix 1
- Data Archive 6.4.3
- Data Archive 6.4 HotFix 2
- Data Archive 6.4 HotFix 1
- Data Archive 6.4
- Data Archive 6.1

Table of Contents

Overview. . . . 2
Types of Data Archive Jobs. . . . 3
Steps for Jobs in Data Archive . . . 3
Steps for Archive Jobs. . . . 3
Steps for Retirement Jobs. . . . 4
Internal Steps of Data Archive Jobs. . . . 5
Online Archive Jobs. . . . 5
Archive Job Requirement. . . . 6
Archive Job Entity Configuration. . . . 7
Archive Job Steps. . . . 10
Archive Jobs with Business Rules. . . . 16
BCP Files. . . . 20
Metadata .xml File. . . . 21
Database Schema and User Details Used. . . . 23
FAQ. . . . 24

Overview

Informatica Archive primarily helps in archiving inactive data present in a source database.

Data Archive jobs can be classified into the following types:

- Online Archive jobs. Database to database. Data is moved from one database to another database.
- Retirement Jobs. Database to Data Vault. Data is moved from a database to Data Vault, a read-only compressed database.

Types of Data Archive Jobs

The following table describes the types of Data Archive jobs:

Job Type	Description
Archive Only	Data moves from source to target. After the job, data is present in both the source and the target used in the job.
Archive and Purge	Data moves from source to target and is then deleted from the source. After the job, data is present in the target and not in the source.
Purge Only	Data is deleted from the source. After the job, data is not available in the source. Generally, Purge Only jobs are scheduled after completion of Archive Only jobs.

Steps for Jobs in Data Archive

Jobs scheduled in the Data Archive Service go through different steps depending on the following conditions:

- Whether the *Use Copy to Staging* option is enabled or disabled in the source connection.
- The type of Data Archive job, i.e., Archive Only, Archive and Purge, or Purge Only.

Steps for Archive Jobs

Archive Jobs are scheduled to move data from one database to another database. Schedule jobs through the **Administration** > **Manage Archive Projects** menu in the Data Archive web user interface.

The following table lists the steps in an Archive job with the *Use Copy to Staging* option enabled and indicates the steps each type of Archive job goes through:

Job Step	Archive Only	Archive and Purge	Purge Only
Generate Candidates	Y	Y	Y
Build Staging	Y	Y	Y
Copy to Staging	Y	Y	Y
Validate Destination	Y	Y	N
Copy to Destination	Y	Y	N
Delete from Source	N	Y	Y
Purge Staging	Y	Y	Y

The following table lists the steps in an Archive job with the *Use Copy to Staging* option disabled and indicates the steps each type of Data Archive job goes through:

Job Step	Archive Only	Archive and Purge	Purge Only
Generate Candidates	Y	Y	Y
Validate Destination	Y	Y	N
Copy to Destination	Y	Y	N
Delete from Source	N	Y	Y
Purge Staging	Y	Y	Y

Steps for Retirement Jobs

Retirement Jobs are scheduled to move data from a database to Data Vault, a read-only compressed database. Schedule jobs through the **Administration** > **Manage Retirement Projects** menu in the Data Archive web user interface. Retirement jobs are Archive Only type jobs. Data is copied from the source to Data Vault and is not purged.

The following table lists the steps in a Retirement job with the *Use Copy to Staging* option enabled and indicates the steps the job goes through:

Job Step	Archive Only
Generate Candidates	Y
Build Staging	Y
Copy to Staging	Y
Validate Destination	Y
Copy to Destination	Y
Delete from Source	N
Purge Staging	Y

The following table lists the steps in a Retirement job with the *Use Copy to Staging* option disabled and indicates the steps the job goes through:

Job Step	Archive Only
Generate Candidates	Y
Validate Destination	Y
Copy to Destination	Y
Delete from Source	N
Purge Staging	Y

Internal Steps of Data Archive Jobs

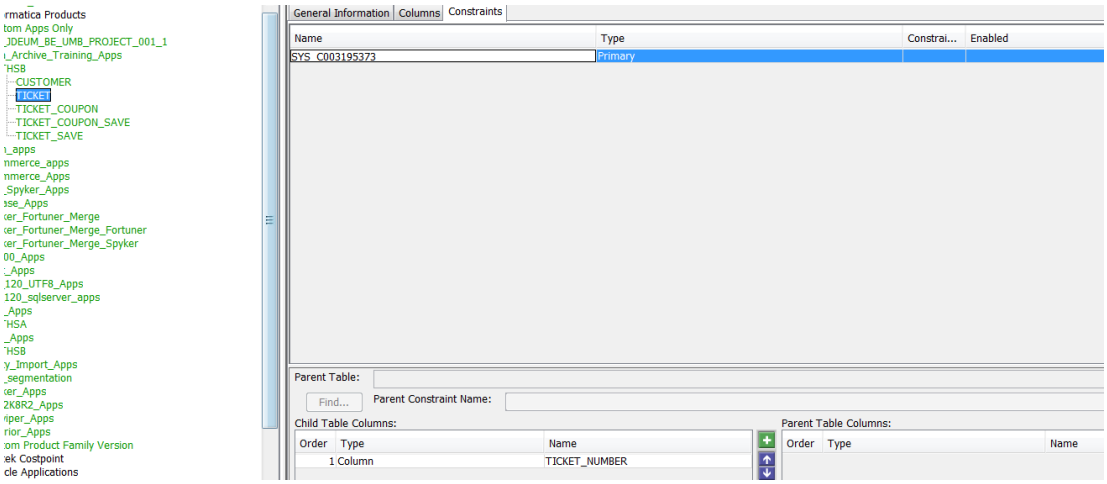
Online Archive Jobs

To understand the queries and actions performed during each of the steps of an Online Archive job (database to database archive job), consider an entity TICKET_001, which has four application tables TICKET, TICKET_COUPON, TICKET_COUPON_SAVE, and TICKET_SAVE.

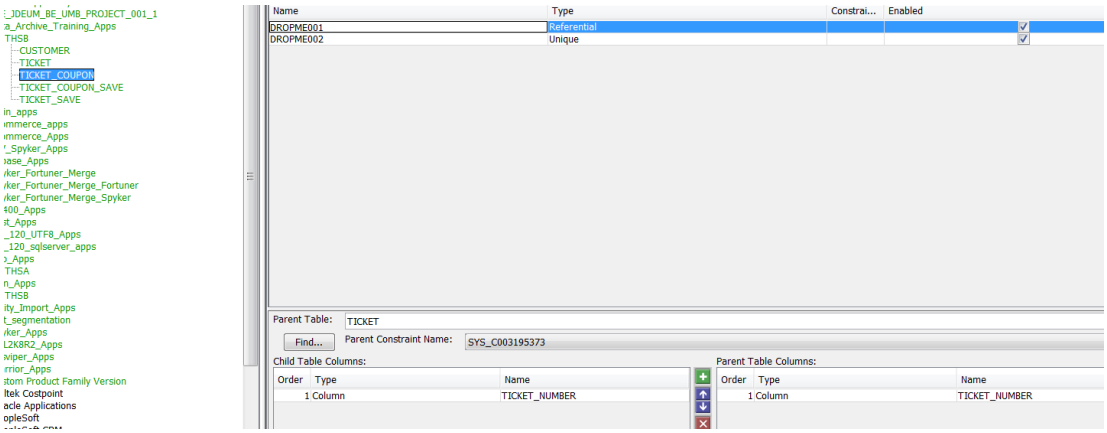
TICKET is the driving table or parent table in the entity. All the other tables in the entity are child tables of TICKET and they are related through information stored in the TICKET_NUMBER column.

Entity Table Constraints

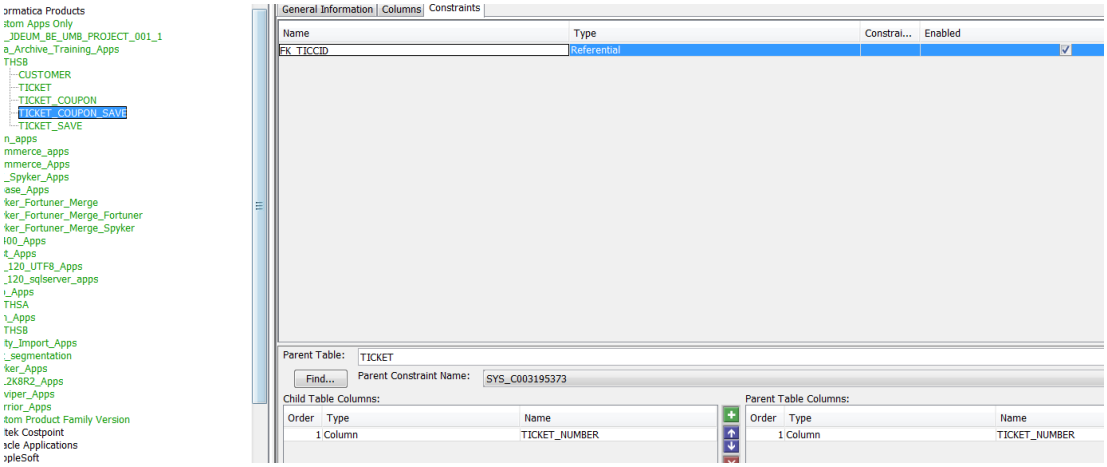
Ticket



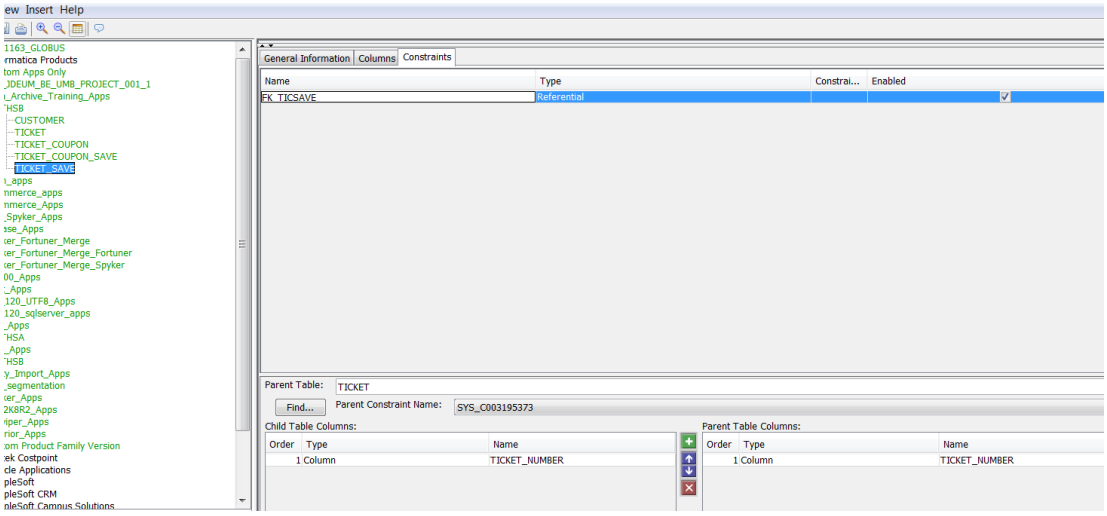
TICKET_COUPON



TICKET_COUPON_SAVE



TICKET_SAVE

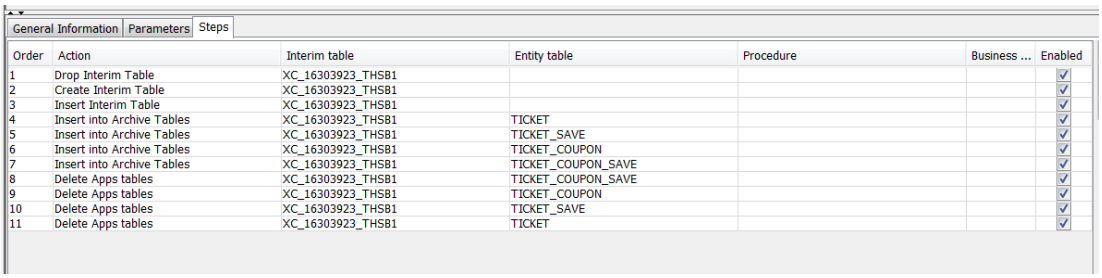


Archive Job Requirement

The Archive job must archive the record with TICKET_NUMBER = 10016995 from the TICKET table and then it must be purged. As the TICKET table is the parent table of the entity, all the referential records for TICKET_NUMBER of 10016995 that are present in the child tables (TICKET_COUPON, TICKET_COUPON_SAVE, and TICKET_SAVE) must also be archived and then purged to maintain data integrity.

Archive Job Entity Configuration

Entity Steps



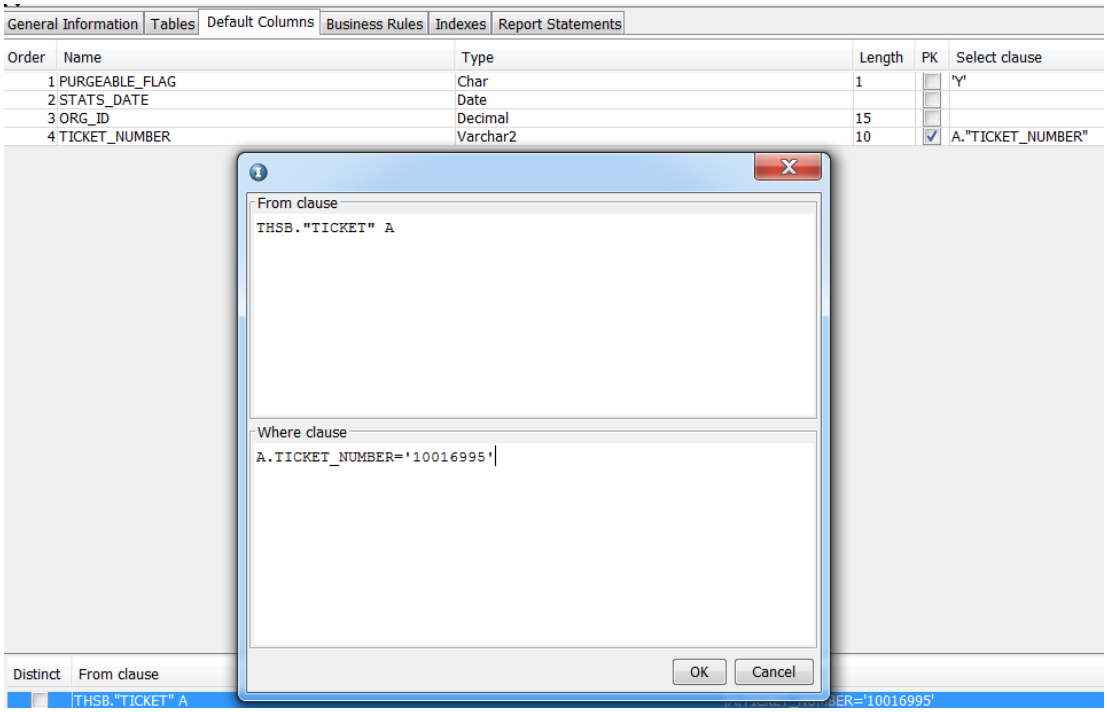
General Information | Parameters | Steps

Order	Action	Interim table	Entity table	Procedure	Business ...	Enabled
1	Drop Interim Table	XC_16303923_THSB1				☑
2	Create Interim Table	XC_16303923_THSB1				☑
3	Insert Interim Table	XC_16303923_THSB1				☑
4	Insert into Archive Tables	XC_16303923_THSB1	TICKET			☑
5	Insert into Archive Tables	XC_16303923_THSB1	TICKET_SAVE			☑
6	Insert into Archive Tables	XC_16303923_THSB1	TICKET_COUPON			☑
7	Insert into Archive Tables	XC_16303923_THSB1	TICKET_COUPON_SAVE			☑
8	Delete Apps tables	XC_16303923_THSB1	TICKET_COUPON_SAVE			☑
9	Delete Apps tables	XC_16303923_THSB1	TICKET_COUPON			☑
10	Delete Apps tables	XC_16303923_THSB1	TICKET_SAVE			☑
11	Delete Apps tables	XC_16303923_THSB1	TICKET			☑

Default Columns

To implement the requirement of archiving the record with 10016995 as the TICKET_NUMBER, the WHERE CLAUSE condition is provided as A.TICKET_NUMBER='10016995' in the ‘Default Columns’ of the interim table.

Default Columns



General Information | Tables | Default Columns | Business Rules | Indexes | Report Statements

Order	Name	Type	Length	PK	Select clause
1	PURGEABLE_FLAG	Char	1	☐	'Y'
2	STATS_DATE	Date		☐	
3	ORG_ID	Decimal	15	☐	
4	TICKET_NUMBER	Varchar2	10	☑	A."TICKET_NUMBER"

From clause

```
THSB."TICKET" A
```

Where clause

```
A.TICKET_NUMBER='10016995'
```

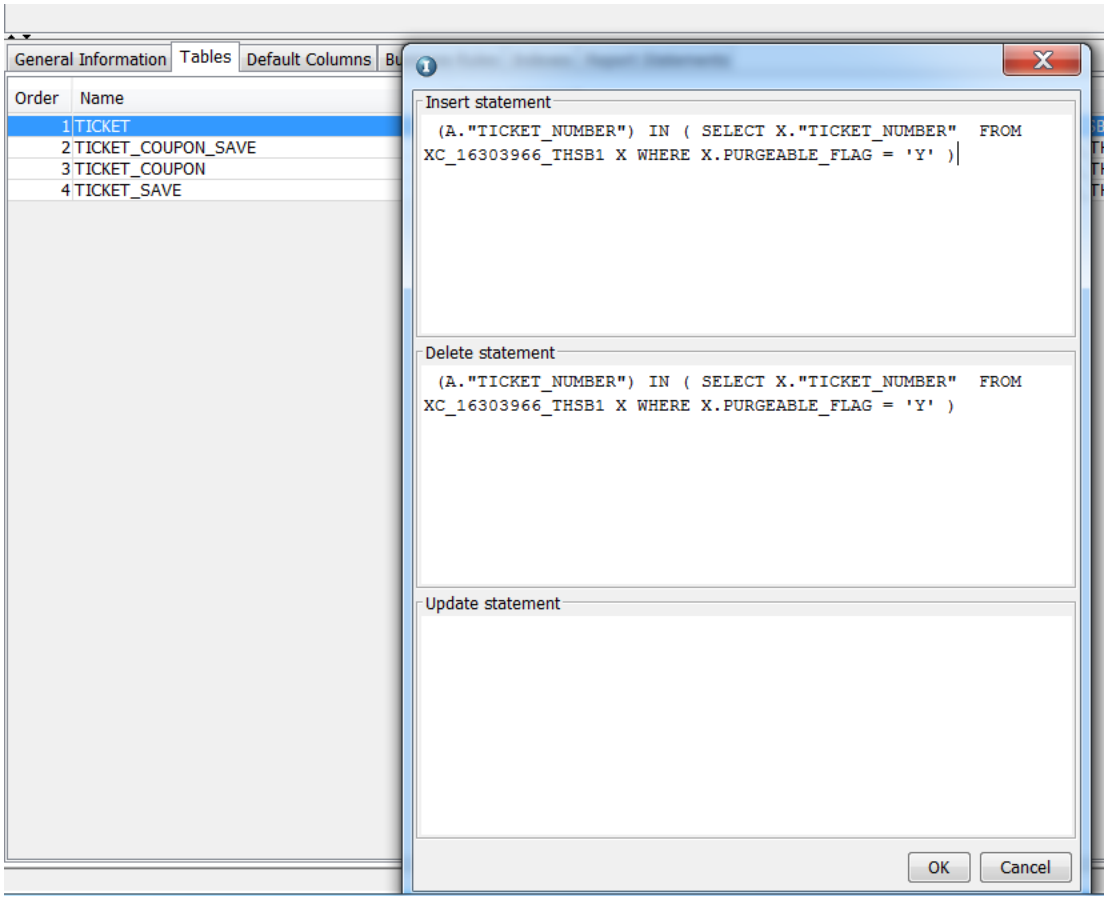
OK | Cancel

Distinct | From clause

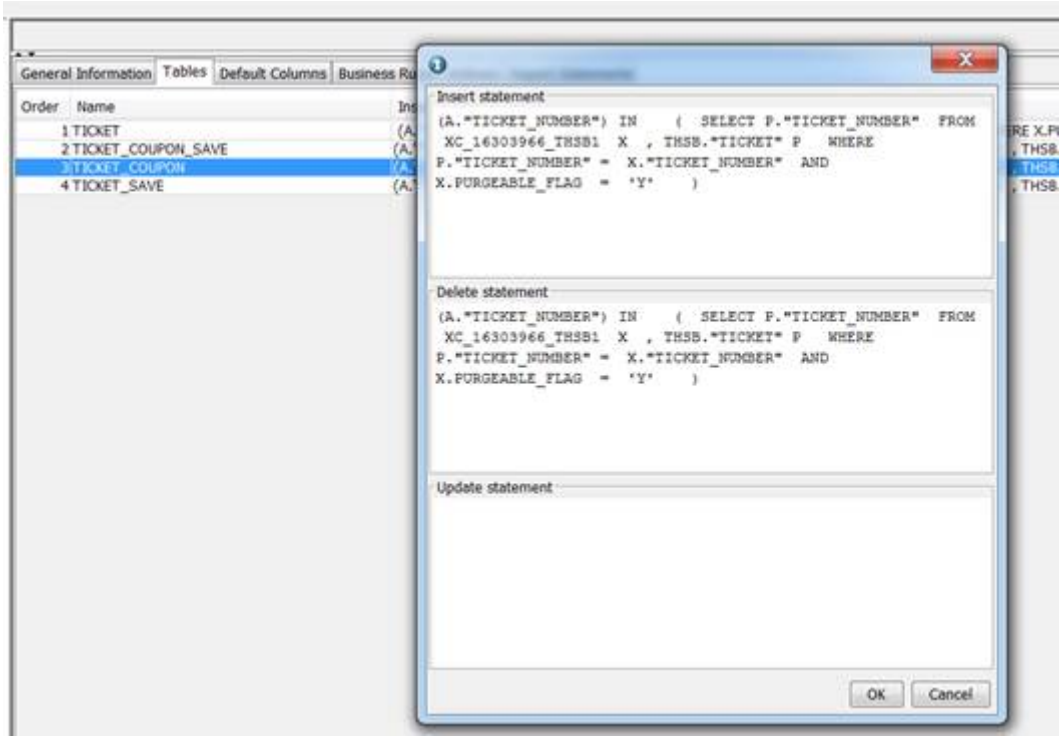
THSB."TICKET" A

Archive Job Insert and Delete Conditions for Tables

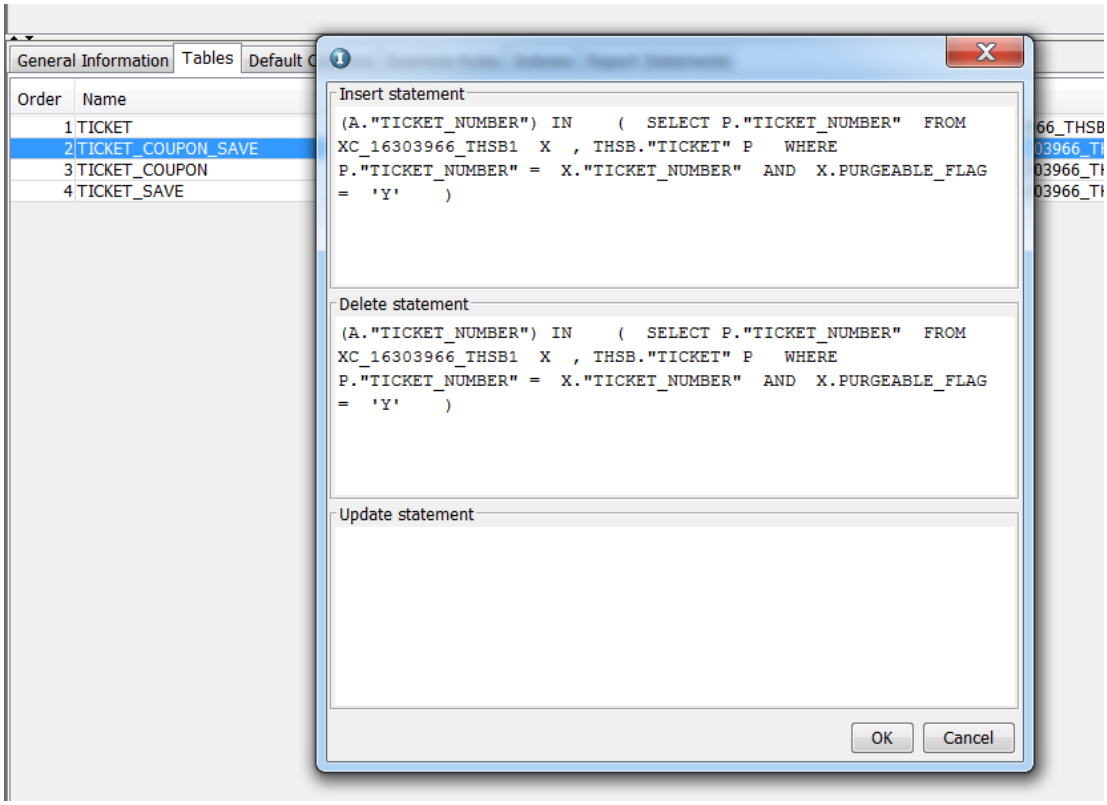
TICKET



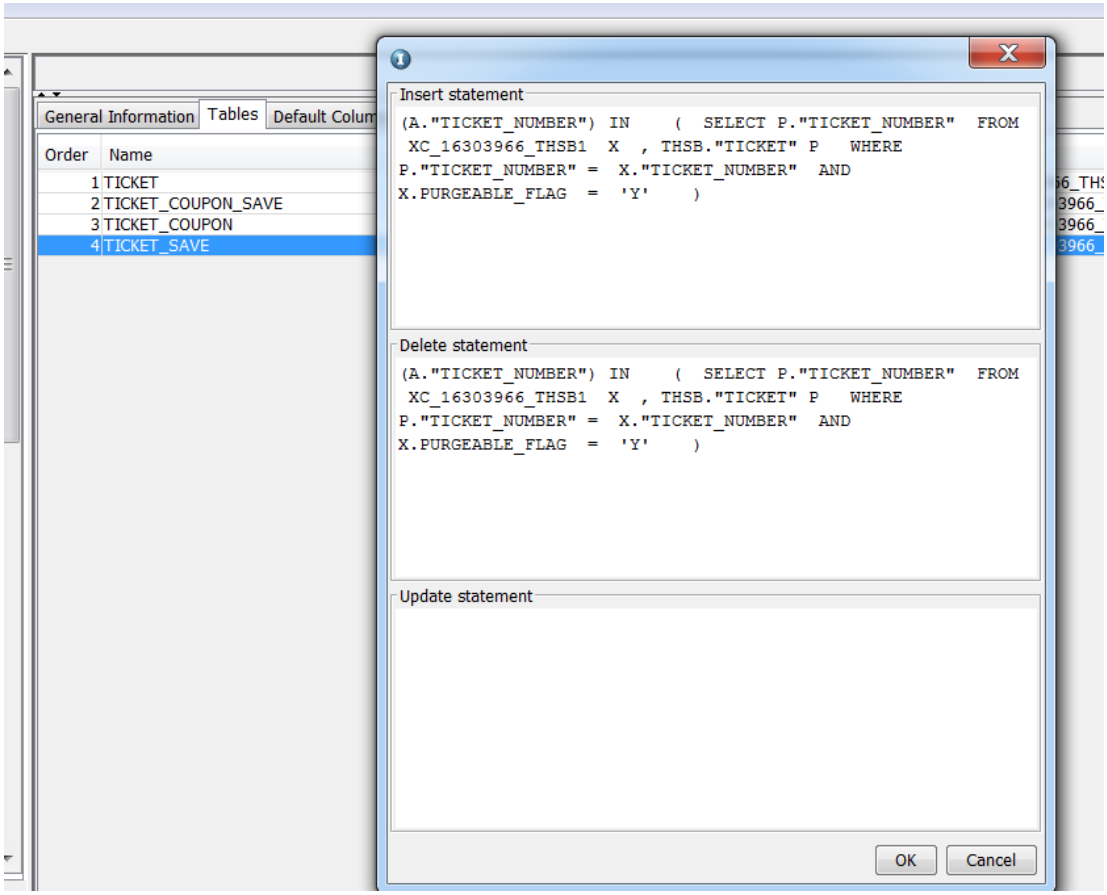
TICKET_COUPON



TICKET_COUPON_SAVE



TICKET_SAVE



Archive Job Steps

The Online Archive job scheduled on the entity is of the type Archive and Purge and the *Use Copy to Staging* option is enabled in the source database connection.

The archive job includes the following steps:

1. Generate candidates
2. Build staging
3. Copy to staging
4. Validate destination
5. Copy to destination
6. Delete from source
7. Purge staging

Generate Candidates

During the Generate Candidates step of the Archive job, an interim table is created at the staging schema of the source database. The staging schema is specified in the source database connection as the 'Staging Schema Name' attribute value.

After creation of the interim table, data is inserted in to the interim table using the source application table name and condition specified in the From clause and the Where clause of the ‘Default Columns’ tab in the interim table.

Naming Convention for the interim table created in the staging schema during the job:

```
[Interim_Table_name_in_entity]_[JOB_ID]
```

Create Interim Table

```
CREATE TABLE [staging_schema]."XC_16303966_THSB1_5"("PURGEABLE_FLAG" CHAR(1
CHAR),"STATS_DATE" DATE,"ORG_ID" DECIMAL(15),"TICKET_NUMBER" VARCHAR2(10
CHAR))
```

Interim table after creation:

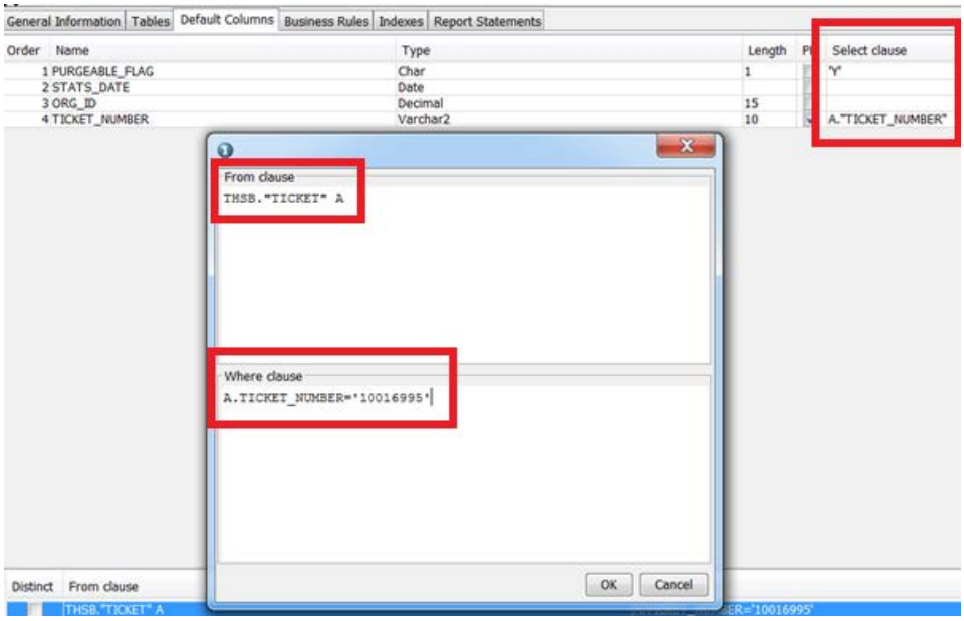
PURGEABLE_FLAG	STATS_DATE	ORG_ID	TICKET_NUMBER

Insert Interim Table

The query that runs during this step inserts data into the interim table created in the staging schema during the job. All records that the auto-generated SELECT query returns using information available in ' Default Columns' are inserted in to the interim table.

The SELECT query is generated with the values specified in the SELECT CLAUSE, FROM CLAUSE, and WHERE CLAUSE in the 'Default Columns' tab of the interim table in the entity.

Select Clause



For the considered entity, the following INSERT query is generated during the step:

```
INSERT /*+ APPEND */INTO [staging_schema]."XC_16303966_THSB1_5"
("PURGEABLE_FLAG", "STATS_DATE", "ORG_ID", "TICKET_NUMBER") SELECT
'Y', null, null, A."TICKET_NUMBER" FROM THSA."TICKET" A WHERE
A.TICKET_NUMBER='10016995'
```

Interim table after insertion:

P.FLAG	STATS_DATE	ORG_ID	TICKET_NUMBER
Y			10016995

Build Staging

During the Build Staging step, temporary or staging tables for each of the application tables in the entity are created in the staging schema of the source database. One staging table is created for each application table in the entity.

Staging schema is specified in the source database connection as the value for the 'Staging Schema Name' attribute. It is the same schema where the interim table is created during the Generate Candidates step.

For the considered entity, four temporary tables are created in the staging schema of the source database since there are four tables in the entity (TICKET, TICKET_COUPON, TICKET_COUPON_SAVE, and TICKET_SAVE).

The temporary tables created in the staging schema have the same structure (the same columns, data type, column length etc.), as the table in the application schema of the source database that it corresponds to.

Naming convention for staging tables:

```
AA_M[Number]_[JOB_ID]
```

Copy to Staging

During the Copy to Staging step, data is inserted into each of the temporary or staging tables with data to be archived from the corresponding application table.

To select data to be inserted into a temporary table, the Insert Condition, specified in the Tables tab of the entity is used.

For the TICKET table

INSERT *+ APPEND */INTO [staging_schema].AA_M234_5(column_list) SELECT (column_list) FROM [source_database_apps_schema].TICKET A WHERE (A."TICKET_NUMBER") IN (SELECT X."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X WHERE X.PURGEABLE_FLAG = 'Y')

	TICKET_NUMBER	ISSUE_DATE	CUST_ID	PASSENGER_NAME	LOCATION	I_ADDED_A_COLUMN	CUP_OF_COFFEE	TICKET	USED
1	10016995	18-OCT-98	370	BARTOLUCCI/TONYA	(null)	(null)	(null)	N	N

For the TICKET_COUPON table:

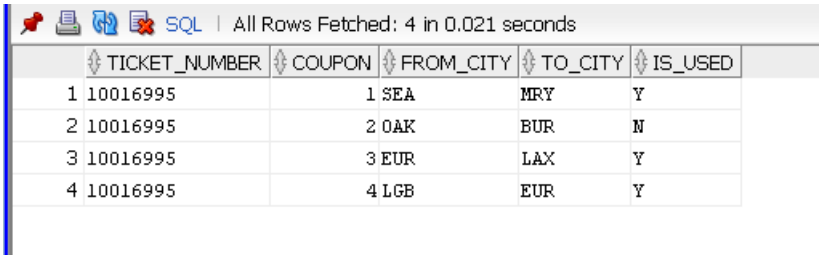
INSERT *+ APPEND */INTO [staging_schema].AA_M235_5(column_list) SELECT (column_list) FROM [source_database_apps_schema].TICKET_COUPON A WHERE (A."TICKET_NUMBER") IN (SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X , [source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND X.PURGEABLE_FLAG ='Y'

SQL | All Rows Fetched: 4 in 0.009 seconds

	TICKET_NUMBER	COUPON	FROM_CITY	TO_CITY	IS_USED
1	10016995	1	SEA	MRY	Y
2	10016995	2	OAK	BUR	N
3	10016995	3	EUR	LAX	Y
4	10016995	4	LGB	EUR	Y

For the TICKET_COUPON_SAVE table:

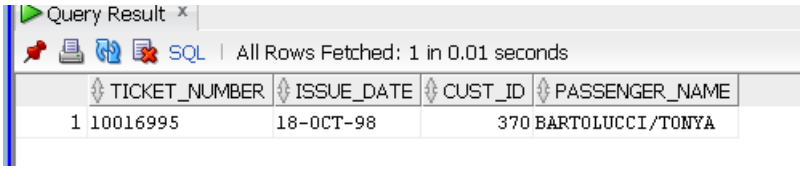
INSERT *+ APPEND */INTO [staging_schema].AA_M236_5(column_list) SELECT (column_list) FROM [source_database_apps_schema].TICKET_COUPON_SAVE A WHERE (A."TICKET_NUMBER") IN (SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X , [source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND X.PURGEABLE_FLAG ='Y')


SQL | All Rows Fetched: 4 in 0.021 seconds

	TICKET_NUMBER	COUPON	FROM_CITY	TO_CITY	IS_USED
1	10016995	1	SEA	MRY	Y
2	10016995	2	OAK	BUR	N
3	10016995	3	EUR	LAX	Y
4	10016995	4	LGB	EUR	Y

For the TICKET_SAVE table:

INSERT *+ APPEND */INTO [staging_schema].AA_M237_5(column_list) SELECT (column_list) FROM [source_database_apps_schema].TICKET_SAVE A WHERE (A."TICKET_NUMBER") IN (SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X , [source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND X.PURGEABLE_FLAG = 'Y')


Query Result

SQL | All Rows Fetched: 1 in 0.01 seconds

	TICKET_NUMBER	ISSUE_DATE	CUST_ID	PASSENGER_NAME
1	10016995	18-OCT-98	370	BARTOLUCCI/TONYA

Sub query	(SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X , [source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND X.PURGEABLE_FLAG = 'Y')
Result	10016995

Validate Destination

During the 'Validate Destination' step, the Data Archive Service verifies whether the application tables being archived already exist in the target database. If the tables do not exist in the target database, they are created.

If the table exists in the target database, then its structure (the number of columns, column name, column data length, data type, etc.,) is validated with the same table in the source database. If any columns are missing in the target database while comparing with the source database, then they are added to the target database.

To verify if a table exists in the target database, run the following query in the target database (Oracle):

```
SELECT COUNT(*) FROM ALL_TABLES T WHERE T.TABLE_NAME = '<table_name_in_upper_case>' AND T.OWNER = '<apps_schema_in_target_database>';
```

To validate the structure of the tables, run the following queries in the source and target databases:

Source database:

```
SELECT COLUMN_NAME, DATA_TYPE, DATA_LENGTH, DATA_PRECISION, DATA_SCALE, NULLABLE, CHAR_LENGTH, CHAR_USED, DATA_TYPE_OWNER FROM ALL_TAB_COLUMNS WHERE TABLE_NAME = '<table_name_in_upper_case>' AND OWNER = '<apps_schema_in_source_database>' ORDER BY COLUMN_ID
```

Target database:

```
SELECT COLUMN_NAME, DATA_TYPE, DATA_LENGTH, DATA_PRECISION, DATA_SCALE, NULLABLE,
CHAR_LENGTH, CHAR_USED, DATA_TYPE_OWNER FROM ALL_TAB_COLUMNS WHERE TABLE_NAME =
'<table_name_in_upper_case>' AND OWNER = '<apps_schema_in_target_database>' ORDER BY
COLUMN_ID
```

Apart from the regular application tables that are part of the entity, the interim table is also created in the target database during this step if not already present. The interim table that is created in the target database does not contain the JOB_ID in the name. The name of the interim table in the target is the same as its name in the entity.

Application tables that are created in the target database contain the following additional column apart from the regular columns in the source database:

```
APPLIMATION_JOB_ID
```

The APPLIMATION_JOB_ID column contains the information about the Data Archive job ID through which data is archived into the target. The information stored in the APPLIMATION_JOB_ID column is used by the Data Archive Service during Restore jobs. The information helps in moving the archived data in the target database back into source database.

In Data Archive versions prior to 6.4, the APP_ENTITY_TABLE_ID column is added to the tables created in the target database apart from the APPLIMATION_JOB_ID column. The APP_ENTITY_TABLE_ID column contains information about the Data Archive entity table ID that is internally used by Data Archive to identify the application table that is part of the entity.

For the considered entity, the interim table is created in the target database as follows:

```
CREATE TABLE [staging_schema]."XC_16303966_THSB1"("PURGEABLE_FLAG" CHAR(1 CHAR),"STATS_DATE"
DATE,"ORG_ID" DECIMAL(15),"TICKET_NUMBER" VARCHAR2(10 CHAR), "APPLIMATION_JOB_ID" NUMERIC NOT NULL)
```

Copy to Destination

During the Copy to Destination step, the Data Archive Service copies data from the staging tables present in the source database into the application tables created in the target database.

For the considered entity, the following queries are run during the Copy to Destination step to select the data to be archived:

```
SELECT <column_list>, <job_id> FROM [staging_schema]."AA_M234_5"

SELECT <column_list>, <job_id> FROM [staging_schema]."AA_M235_5"

SELECT <column_list>, <job_id> FROM [staging_schema]."AA_M236_5"

SELECT <column_list>, <job_id> FROM [staging_schema]."AA_M237_5"

SELECT <column_list>, <job_id> FROM [staging_schema]."XC_16303966_THSB1_5"
```

Records returned by the above SELECT queries are inserted into the corresponding table at the application schema in the target database.

If you do not enable the *Use Copy to Staging* option in the source connection, data is copied directly from the application schema in the source database into the application schema in the target database. When staging is not available for selecting data to be archived into the target application table, the insert condition specified in the Tables tab of the entity is used in the SELECT query executed at the source database.

If you do not enable the *Use Copy to Staging* option in the source connection for the considered entity, the following queries are executed during the Copy to Destination step to select the data to archive:

```
SELECT (column_list),<job_id> FROM [source_database_apps_schema].TICKET A WHERE (A."TICKET_NUMBER")
IN ( SELECT X."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X WHERE X.PURGEABLE_FLAG =
'Y' )

SELECT (column_list),<job_id> FROM [source_database_apps_schema].TICKET_COUPON A WHERE
(A."TICKET_NUMBER") IN ( SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X ,
[source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND
X.PURGEABLE_FLAG = 'Y' )

SELECT (column_list),<job_id> FROM [source_database_apps_schema].TICKET_COUPON_SAVE A WHERE
(A."TICKET_NUMBER") IN ( SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X ,
[source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND
X.PURGEABLE_FLAG = 'Y' )

SELECT (column_list),<job_id> FROM [source_database_apps_schema].TICKET_SAVE A WHERE
(A."TICKET_NUMBER") IN ( SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X ,
[source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND
X.PURGEABLE_FLAG = 'Y' )
```

Delete from Source

During the Delete from Source step, the Data Archive Service deletes data from the application tables present in the source database.

To select data to delete from the application table, the Delete Condition specified in the Tables tab of the entity is used.

For the considered entity, the following statements are run during the Delete from Source step:

```
DELETE FROM [source_database_apps_schema]."TICKET_COUPON" A WHERE (A."TICKET_NUMBER") IN (SELECT
P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X ,
[source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND
X.PURGEABLE_FLAG = 'Y' )

DELETE FROM [source_database_apps_schema]."TICKET_COUPON_SAVE" A WHERE (A."TICKET_NUMBER") IN
(SELECT P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X ,
[source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND
X.PURGEABLE_FLAG = 'Y')

DELETE FROM [source_database_apps_schema]."TICKET_SAVE" A WHERE (A."TICKET_NUMBER") IN (SELECT
P."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X ,
[source_database_apps_schema]."TICKET" P WHERE P."TICKET_NUMBER" = X."TICKET_NUMBER" AND
X.PURGEABLE_FLAG = 'Y' )

DELETE FROM [source_database_apps_schema]."TICKET" A WHERE (A."TICKET_NUMBER") IN (SELECT
X."TICKET_NUMBER" FROM [staging_schema].XC_16303966_THSB1_5 X WHERE X.PURGEABLE_FLAG = 'Y' )
```

Purge Staging

During the Purge Staging step, the interim tables (created during the Generate Candidates step) and the temporary staging tables for application tables (created during the Build Staging step) are dropped.

For the considered entity, the following statements are run during the Purge Staging step:

```
DROP TABLE "<staging_schema>"."XC_16303966_THSB1_5" PURGE

DROP TABLE "<staging_schema>"."AA_M234_5" PURGE

DROP TABLE "<staging_schema>"."AA_M235_5" PURGE

DROP TABLE "<staging_schema>"."AA_M236_5" PURGE
```

```
DROP TABLE "<staging_schema>"."AA_M237_5" PURGE
```

Archive Jobs with Business Rules

Business Rules help handle the exceptions to a condition used for archiving. Business Rules are evaluated in the Generate Candidates step.

Entity and Business Rule

To understand the evaluation of Business Rule during an archive job, consider an entity ORDERS_001 that has one application table ORDERS.

Use Case

Consider the following example scenario to create a Business Rule in the entity:

Basic condition for archiving	Order is delivered and it is older than 30 days
Exception to the base condition	IS_USED='Y', if the order is being used by the other teams and it should not be archived IS_USED='N', the order is not used and it can be archived.

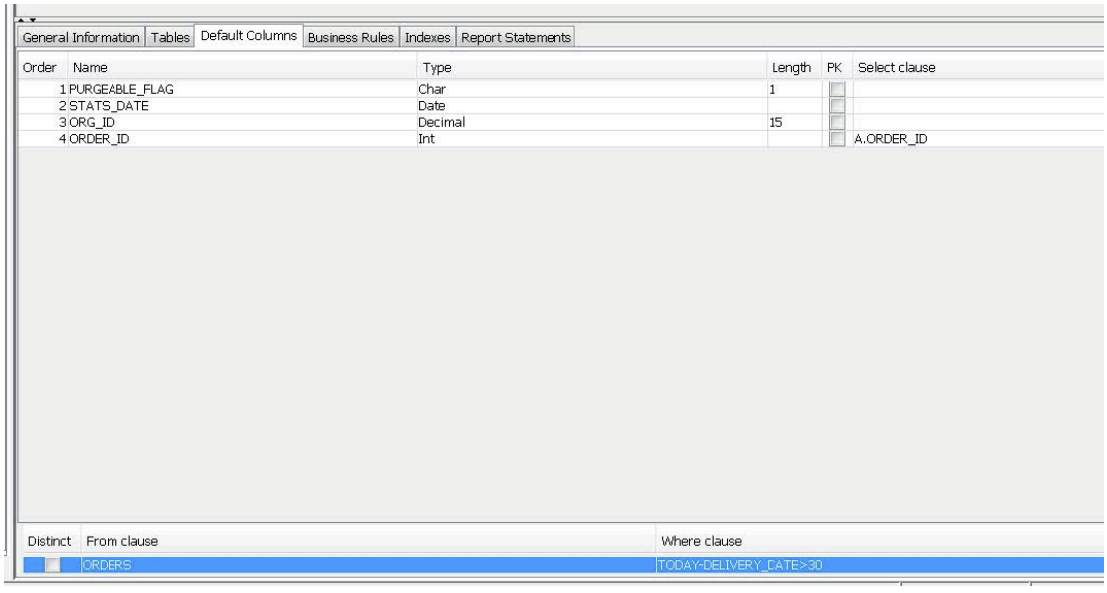
Business Rule

The following conditions are used in the entity to implement the use case:

Basic condition for archiving	TODAY-DELIVERY_DATE>30
Business rule name	IS_USED_BUSINESS
Business rule condition	ORDER_ID IN (SELECT ORDER_ID FROM ORDERS WHERE IS_USED='Y')

The base condition for archiving is specified in the Where clause field of the 'Default Columns' tab in the interim table.

The business rule name and condition are defined in the Business Rules tab of the interim table. In the given scenario, the ORDER_ID column used to the left of the IN clause in the Business Rule condition must be present in the 'Default Columns' tab of the interim table.



Source Application Table (ORDERS)

Sample data in the Orders table that is older than 30 days:

ORDER_ID	IS_USED	Quantity
123	'Y'	2
124	'N'	3
125	'N'	4

Actions Performed During Job Steps

Generate Candidates

Create Interim Table

Interim table after creation:

P.FLAG	STATS_DATE	ORG_ID	ORDER_ID	IS_USED_BUSINESS

Insert Interim Table

```
INSERT INTO [staging_schema].[Interim_table] SELECT '', '','', A. ORDER_ID,'' FROM
[source_database_apps_schema].ORDERS A WHERE A.TODAY-DELIVERY_DATE>30;
```

Interim table after execution of above query:

P.FLAG	STATS_DATE	ORG_ID	ORDER_ID	S_USED_BUSINESS
			123	
			124	
			125	

Update Interim Table

First Update Query

```
UPDATE [staging_schema].[interim_table] A SET A.IS_USED_BUSINESS =1 WHERE A.ORDER_ID IN (SELECT
ORDER_ID FROM [source_database_apps_schema].ORDERS WHERE IS_USED='Y')
```

Sub query	(SELECT ORDER_ID FROM ORDERS WHERE IS_USED='Y')
Result	123

Final Simplified Query

```
UPDATE [staging_schema].[interim_table] A SET A.IS_USED_BUSINESS =1 WHERE ORDER_ID IN (123)
```

Data in interim table after the first update query:

P.FLAG	STATS_DATE	ORG_ID	ORDER_ID	IS_USED_BUSINESS
			123	1
			124	
			125	

Second Update Query

```
UPDATE [staging_schema].[interim_table] A SET A.IS_USED_BUSINESS=0 WHERE A.IS_USED_BUSINESS IS NULL
```

Data in interim table after second update query:

P.FLAG	STATS_DATE	ORG_ID	ORDER_ID	IS_USED_BUSINESS
			123	1
			124	0
			125	0

Third Update Query

```
UPDATE [staging_schema].[interim_table] A SET A.PURGEABLE_FLAG='Y' WHERE COALESCE(IS_USED_BUSINESS, 0)= 0
```

Data in interim table after third update query:

P.FLAG	STATS_DATE	ORG_ID	ORDER_ID	IS_USED_BUSINESS
			123	1
'Y'			124	0
'Y'			125	0

Fourth Update Query

```
UPDATE [staging_schema].[interim_table] A SET A.PURGEABLE_FLAG='N' WHERE COALESCE(IS_USED_BUSINESS, 1)=1
```

Data in interim table after fourth update query:

P.FLAG	STATS_DATE	ORG_ID	ORDER_ID	IS_USED_BUSINESS
'N'			123	1
'Y'			124	0
'Y'			125	0

P.FLAG	Y - Record can be archived
P.FLAG	N- Record should not be archived

Copy to Destination

Query to select data to archive from the application table ORDER:

```
SELECT [column_list] FROM ORDERS A WHERE (A."ORDER_ID") IN (SELECT X."ORDER_ID" FROM XC_4_THSA1_5 X
WHERE X. PURGEABLE_FLAG = 'Y')
```

Sub query	(SELECT X."ORDER_ID" FROM XC_4_THSA1_5 X WHERE X. PURGEABLE_FLAG = 'Y')
Result	124,125

Final simplified query to select records for archiving:

```
SELECT [column_list] FROM ORDERS A WHERE (A."ORDER_ID") IN (124,125);
```

Hence only the records with Order ID of 124 and 125 are archived into the target. The record with Order ID 123 is not archived into the target even though it is older than 30 days because it satisfies the exception defined in the business rule.

Retirement Jobs

Data Vault, also called as File Archive Service (FAS), is a read-only compressed database that is used along with the Data Archive Service for Retirement projects.

Data Archive archives data into Data Vault through two jobs, the Extraction Job and the Crawler Job (LoaderJob).

A Data Archive Extraction job extracts data from the source database into a flat file of type BCP. The Loader Job then reads the BCP files created by the Extraction Job and loads the data from them into Data Vault.

During the Loader Job, flat file data from the source database is compressed and then loaded into tables in Data Vault. Compressed data files that contain the actual data are generated for all the tables archived into Data Vault. Compressed data files in Data Vault are in SCT format.

Extraction Job

An Extraction Job scheduled as part of the retirement project fetches data from the source database and creates temporary BCP files with the data. BCP files are created in the staging location defined in the Data Vault target connection.

For the extraction job created from the retirement project, the source and target are as follows:

Source	The source database referred by the source connection used in the retirement project.
Target	The staging location used in the Data Vault target connection used in the retirement project.

Generate Candidates

During this step no action is performed in a Retirement job because the Drop Interim Table, Create Interim Table, and Insert Data into Interim Table steps are disabled in the retirement entity.

Validate Destination

During this step a check is made to see if the staging location is accessible from the Data Archive server machine. If the staging location does not exist or if is not reachable from the Data Archive server machine, the job returns an error. Retirement jobs do not create the staging location. An existing folder location is needed to use as the staging location to run the job.

Copy to Destination

During this step the job creates the temporary files (BCP Files) in the staging location for each of the application tables present in the entity used with the retirement job. For the extraction job scheduled through the retirement project, the staging location is the target and hence the Copy to Destination step creates BCP files in the staging location. BCP files are created with data obtained from application tables in the source database. The condition used to get the data is defined in the Tables tab for the INSERT statement in the entity.

Generally, for a Retirement Job, the INSERT statement in the entity would be 1=1, which means that all data from the table is extracted and written into the BCP file. For example, for the application table TICKET in the retirement entity with INSERT statement as '1=1', the following query is issued to get the data from the source database:

```
SELECT (column_list) FROM [source_database_apps_schema].TICKET A WHERE 1=1
```

Purge Staging

During this step, no action is performed because no interim table is created during the Generate Candidates step and *Use Copy to Staging* is not used in the source connection of a Retirement Job. This step does not delete any BCP files in the staging location of the Data Vault target connection.

BCP Files

BCP files are similar to comma separated values (CSV) files. They have data extracted from the source database for application tables that are part of the entity to be archived.

Naming convention:

```
[EXTRACTION_JOB_ID]_M[Number]_[META_TABLE_ID]_[yyyymmdd_at_which_bcp_file_has_been_created]_[sequence_id_of_bcp_file_for_table].bcp
```

BCP does not contain column header information (column names) for the tables from which data has been extracted. It contains only the data.

BCP contains two types of delimiters for every record in it:

Row Delimiter	Used to distinguish one record from another record present in the table. Default 'Row Delimiter' is '%#%#'.
Column Delimiter	Used to distinguish one column value from another column value, present within the same record. Default column delimiter is '%@%'.

All BCP files created from a particular extraction job must have the same row and column delimiters.

To change a delimiter used in the BCP files, you must update the following properties in the conf.properties file in the <ILM_HOME> location.

Use the following command to allow delimiters in BCP files to have different characters:

```
#informia.bcp.columnSeparator=%@%
```

```
#informia.bcp.rowSeparator=%#%#x
```

By default, each BCP file contains one million records. The number of BCP files created for an application table depends on the number of records to be archived for the table. If the table has less than one million records to be archived then only one BCP file is created.

In general, you can use the following formula to calculate the number of BCP files created for an application table:

Number of BCP Files = (Number of records to be archived/1 Million) +1

For example, if the number of eligible records to archive in an application table is 2,400,000, then the number of BCP files created is:

```
Number of BCP Files = 2,400,000/1,000,000+1 = 2+1
```

```
Number of BCP Files = 3
```

Metadata .xml File

At the completion of an extraction job, the Metadata.xml file is generated at the staging location specified in the Data Vault target location in addition to the BCP files for each of the application tables.

Naming Convention:

```
[EXTRACTION_JOB_ID]_[yyyymmdd_at_which_xml_filehas_been_created]_1.xml
```

The Metadata.xml file contains information about BCP files created during the extraction job and metadata information for each of the Data Archive repository schema tables like AM_META_TABLE, AM_META_COLUMNS, AM_REP_TABLE_COLUMNS, XA_ENTITIES, etc.

The following table lists the tags in the Metadata.xml file that are related to the metadata information stored for Data Archive repository schema tables:

Tag	Description
<TABLE>	Specifies the start of information corresponding to the repository table. The name of the table to which the information corresponds is included in the attribute NAME inside the tag. For example, consider the following: <TABLE NAME="AM_DATA_REPOSITORY">. The tag specifies the start of information corresponding to the repository table AM_DATA_REPOSITORY. <TABLE> contains other sub tags like <ROWS> and <ROW>.
<ROWS>	Specifies the start of records corresponding to the table referred by its parent tag <TABLE>. <ROWS> contains another sub tag <ROW>. One sub tag <ROW> is present for each of the records in the table. If a table has 100 records, then there are 100 <ROW> sub tags inside the <ROWS> tag.
<ROW>	Specifies the actual record corresponding to the table. It is a sub tag for <ROWS>, which in turn is a sub-tag of <TABLE>. *<ROW>* contains another sub tag *<COLUMN>* . One sub tag *<COLUMN>* is present for each of the columns in the records of a table.
<COLUMN>	Specifies the column information for a record. It is a sub tag of <ROW>, which is a sub tag of <ROWS>. Name of the column which it corresponds to is referred by the attribute NAME part of the tag. The CDATA attribute in the tag specifies the actual value of the field.
</COLUMN>	Specifies the end of column related information for the active record, referred by the <ROW> tag.

Tag	Description
</ROW>	Specifies the end of information for one complete record for a table, referred by its grandparent tag <TABLE>.
</ROWS>	Specifies the end of information regarding all records for a table, referred by its parent tag <TABLE>.
</TABLE>	Marks the completion of the information corresponding to the repository table.

For example, consider the following content in the Metadata.xml file for one of the repository tables:

```
<TABLE NAME="AM_DATA_REPOSITORY">
<ROWS>
<ROW>
<COLUMN NAME="REP_ID"><![CDATA[5]]></COLUMN>
<COLUMN NAME="PLATFORM_ID"><![CDATA[51]]></COLUMN>
<COLUMN NAME="PRODUCT_FAMILY_VERSION_ID"><![CDATA[-26]]></COLUMN>
<COLUMN NAME="REP_NAME"><![CDATA[SQLServer_Source_Test]]></COLUMN>
<COLUMN NAME="REP_DESC"><![CDATA[infa@123]]></COLUMN>
<COLUMN NAME="REP_TYPE_ID"><![CDATA[1]]></COLUMN>
<COLUMN NAME="PRODUCT_ID"><![CDATA[3]]></COLUMN>
</ROW>
<ROW>
<COLUMN NAME="REP_ID"><![CDATA[8]]></COLUMN>
<COLUMN NAME="PLATFORM_ID"><![CDATA[52]]></COLUMN>
<COLUMN NAME="PRODUCT_FAMILY_VERSION_ID"><![CDATA[-26]]></COLUMN>
<COLUMN NAME="REP_NAME"><![CDATA[IDV_Spyker_Target_Connection]]></COLUMN>
<COLUMN NAME="REP_DESC"><![CDATA[]]></COLUMN>
<COLUMN NAME="REP_TYPE_ID"><![CDATA[2]]></COLUMN>
<COLUMN NAME="PRODUCT_ID"><![CDATA[3]]></COLUMN>
</ROW>
</ROWS>
</TABLE>
```

For the above XML entries of the table AM_DATA_REPOSITORY, the following table lists the table structure and data in the Data Archive repository database schema:

REP_ID	PLATFORM_ID	PRODUCT_FAMILY	REP_NAME	REP_DESC	REP_TYPE_ID	PRODUCT_ID
5	51	-26	SQLServer_Source_Test	infa@123	1	3
8	52	-26	IDV_Spyker_Target_Connection		2	3

Database Schema and User Details Used

Online Archive Jobs (Database to Database)

This section lists the database details for the Archive jobs and the Retirement jobs mentioned in the document.

Archive Jobs with Staging

Step	Connection Used	Table type (type of tables created)	Schema (Schema where temporary/ application tables are created)	Login (User details used to query data and to insert data into the temporary/ application tables)
Generate Candidates	Source	Temporary -> Interim Table (XC_1_)	Staging Schema Name	Staging Login Details
Build Staging	Source	Temporary -> (AA_M_)	Staging Schema Name	Staging Login Details
Copy to Staging	Source	Temporary -> (AA_M_)	Staging Schema Name	Staging Login details
Validate Destination	Target	Application --> Actual tables would be created	Application Schema	Application Login details
Copy to Destination	Target	Application --> (actual tables. E.g. TICKET, TICKET_COUPON etc..)	Application Schema Name	Application Login Details
Delete from Source	Source	N/A (Tables will not be created. Deletion of data from Actual Application table will happen)	Application Schema	Application login details

Archive Jobs without Staging

Step	Connection Used	Table type (type of tables created)	Schema (Schema where temporary/ application tables are created)	Login (User details used to query data and to insert data into the temporary/ application tables)
Generate Candidates	Source	Temporary -> Interim Table (XC_1_)	Staging Schema Name	Staging Login Details
Validate Destination	Target	Application --> Actual tables would be created	Application Schema	Application Login details
Copy to Destination	Target	Application --> (actual tables. Example, TICKET, TICKET_COUPON, etc.)	Application Schema Name	Application Login Details
Delete from Source	Source	N/A (Tables will not be created. Deletion of data from Actual Application table will happen.)	Application Schema	Application login details

Retirement Jobs

Step	Connection Used	Table type (type of tables created)	Schema (Schema where temporary/ application tables are created)	Login (User details used to query data and to insert data into the temporary/ application tables)
Generate Candidates	Source	NA	NA	NA
Validate Destination	Target	NA	NA	NA
Copy to Destination	Target	BCP File for each Application table --> (one BCP file for each 1 Million records from actual application tables. E.g. TICKET, TICKET_COUPON etc.)	NA. BCP file would be created in the 'Staging Directory' provided in Target Connection).	Application Login Details from source connection.

FAQ

Should the *Use Copy to Staging* option be used in a source connection created for Retirement jobs?

When the 'Use Copy to Staging' option is enabled, the job includes two additional steps - Build Staging and Copy to Staging in a Retirement job, apart from the standard steps.

The Staging option is useful only for jobs of type Archive and Purge. That is, jobs which involve the Delete from Source step. In the Archive and Purge type jobs, if you use the staging option, copying data into the target and deletion of data from the source are performed in parallel. This helps in improving the performance of the job.

Retirement jobs are of the type Archive Only. Data is copied into the target and data from the source database is not deleted. Because of this, the 'Use Copy to Staging' option is redundant. In addition, the 'Use

Copy to Staging' option might increase the Retirement job cycle time since there would be an additional data copy operation from source to temporary table (Copy to Staging step) and then from temporary table to creation of BCP file at the staging location specified in the Data Vault target connection (Copy to Destination step).

Which table metadata information in the Metadata.xml file is used to compare and validate the structure of a table in Data Vault?

When a loader job is scheduled, as a first step the tables are created if the tables are not already available in the archive folder of Data Vault. If the tables are present already in Data Vault, then the current structure of the tables obtained from the source database during the extraction job is validated with the structure present in Data Vault.

If a column has been added to a table or if the size of any of the columns in the table has increased in the source database since the last loader job, the changes are made to the corresponding table in Data Vault.

During a loader job, no connection is made to the source database to get the table's metadata information. The loader job uses the metadata information available in the Metadata.xml file for all table DDL related activities such as creation, validation, alteration, etc. in Data Vault.

The metadata information of source tables is present in two sections, AM_META_COLUMNS and AM_REP_TABLE_COLUMNS, of the Metadata.xml file. Data in these sections is obtained from the corresponding tables present in the Data Archive repository database schema during the Copy to Destination step of a retirement job.

AM_META_COLUMNS might not have the latest changes in the source database after importing the table metadata in EDM. This means that if any columns have been added or the data length for the column in the source database has been modified recently but not imported into EDM, the information would not be available in the AM_META_COLUMNS table.

Metadata information for every table in the entity is dynamically obtained from the source database each time a retirement job is scheduled by the Data Archive Service. Changes made in a table in the source database after importing into EDM are obtained during the retirement job. Recent metadata information of tables obtained during retirement jobs is stored in the repository table AM_REP_TABLE_COLUMNS.

Therefore the AM_REP_TABLE_COLUMNS table in the repository database contains the latest metadata information about the tables. As AM_REP_TABLE_COLUMNS has the correct metadata information, the metadata information stored in the Metadata.xml file for AM_REP_TABLE_COLUMNS section is used to perform structural validation between tables in the source database and Data Vault.

Author

Thirumurugan Swaminathan

Acknowledgements

The author would like to acknowledge Vallinayagam Alwar and Nandakumar Paramasivam for their technical assistance.